Reference Story No.	Pool	Story Source	Story Summary
1	1	EECS 168 SP25	Wrote a Python program that mimics a web browser. The program tracks the current url/page and allows navigating to others. Functionality for history and going back a page.
2	1	EECS 348 FA25	Developed C program that when given an integer responded the possible ways an American Football team could have scored it in their game. (# Touchdowns, td with xp, td with 2ptc, fg, safety)
3	1	Personal Project FA26	A website where you build a football team by rolling pokemon and choosing their position had its code public. Used it to make a python script to tell you where to put what pokemon based on stats and position weight.
4	1	EECS 168 FA22	Wrote a Python program that when given an integer representing number of soda cans gives back the combination of 24 packs, 6 packs and singles to get the wanted quantity.
5	2	EECS 348 FA25	Wrote a C++ program which implements several matrix operations such as swapping cols, rows, multiplication, summing diagonals. Reads in matrices from external file and verifies validity of size vs operation.
6	2	EECS 581 FA26	Added timer functionality to existing Minesweeper code with Python tk library. When enabled, the timer counted down from a time provided by the user (max 1hr).
7	2	EECS 330 SP26	Implemented mergesort in Python. A predefined list would be repeatedly split in half and swapping elements until the list returned is sorted from least to greatest.
8	2	EECS 468 SP26	Created a program in Javascript with various simple projects such as a loop that incremented by 1 to output a triangle made of '*', number between 1-100 except those that are divisible by 7 or 4, or both, and an 'n-by-n' grid. Outputs were displayed on an HTML page.
9	3	EECS 468 SP26	Implemented Nim in Haskell. Two players take turns taking however many items they wish from defined piles. The last player to take the last item from the last pile loses.
10	3	EECS 268 SP25	Developed a Python program for EECS 268 that simulated a CPU Scheduler. Input was a text file which had commands, program tracked the mock processes with both linked stack and linked queue.
11	3	EECS 565 SP26	Implemented a buffer overflow attack following instructions from a SEED Lab, which involved crafting an input that would exploit a lack of boundary checks in a program.
12	5	EECS 468 FA25	Wrote a series of functions in haskell for combinatorics, some basic coding functions and some mathematical equations. Included distributing objects into boxes changing what is distinguishable, list operations, n choose k etc.
13	5	EECS 565 FA26	Vigenere Cipher encrypt/decrypt algorithm implemented in Python that uses a ciphertext (or plaintext for encryption), key length, and first word length. It brute-force decrypts the text by comparing the first word with a stored word dictionary and outputs plaintext.
14	5	EECS 468 SP26	Implemented JavaScript server that supported GET, PUT, DELETE, and MKCOL actions through HTTP requests within the terminal. Allowed user the create, delete, and update files using the server.
15	8	EECS 510 FA25	Designed a unique language, defined its formal grammar and constructed a pushdown automaton based on states, transitions and stack operations that recognizes the created language based on accepting/rejecting strings.
16	8	EECS 388 SP26	Developed and implemented a simulated automatic braking system (in C/C++) which uses LIDAR connected to one board which communicates to another board with LED to change color or flash red depending on distance.
17	8	EECS 348 FA25	Developed and documented a program to generate a truth table based on over 20 standard boolean properties. Made class and use case diagrams to document the program with UML criteria.
18	13	Personal project FA26	Created an AI draft assistant for the PC game League of Legends, utilizing personal or team stats to suggest champion picks for collegiate draft. Draws from a database of inputted scrims, matches, and ranked games to predict best picks based on previously drafted and banned champions.
19	13	EECS 348 SP26	Implemented a simple compiler/calculator in C++ that could be interacted with by a user to do basic operations such as +,-,/, and // while matching parentheses and parsing tokens.
20	13	ASTR 591 FA25	Wrote a python notebook which got star cluster data from class materials and trimmed stars based on properties. Used numpy to graph stars based on brightness and color to infer about the cluster.
21	21	EECS 568 SP26 - Data Mining Final Project	Utilized machine learning to build an advanced deep learning neural network implementing RETAIN to predict patient mortality based on previous medical data including diagnoses, procedures, and prescriptions.
22	21	HackKU25 - MovieOracle	Users can search media titles for AI-generated, opinion-based recommendations and track what they've watched, plan to watch, and rated. Uses a third party API for IMDB and Gemini scrapes the reviews.
23	21	HackKU26 - BetOnMe	Goal/Accountability website. Users stake XRP on personal goals, verify attendance with geotagged selfies, and donate their locked stake to charity if they miss a scheduled session.

The initial requirements are on Sheet2

Link to example reqs

https://docs.google.com/spreadsheets/d/1TM_8j_1qYAN1xsiGHg9kv8SyZwMbijuF7QhaUY9nWBE/edit?gid=24592193O#gid=245921930

Req ID	Description	Story Pts	Priority	Sprint No.
	Map			
1	Map generator builds 8-10 tiers with 3-4 lanes from a seed, forward-only connections, no crossing paths, and a single boss node at the top.	13	1	
2	Path validation and repair ensures no orphaned nodes, every node is reachable from a start node, and every node leads to the boss.	8	2	
3	Map nodes are assigned a type (fight, shop, rest) by weighted rolls with constraints (at least one shop, a rest node always precedes the boss, no two shops in a row).	5	3	
4	Map screen displays the generated nodes and connections in a vertical, Slay the Spire style layout, with the start at the bottom and the boss at the top.	5	4	1
5	Map UI highlights reachable nodes, marks visited nodes, and shows an icon for each node type.	3	5	1
6	Swordsman sprite travels along connections to a selected next node and cannot move backward.	3	6	1
7	Map seed is displayed on screen and the same seed regenerates the same map.	1	55	
8	Shop node sells relics and a full heal for gold.	3	37	
9	Rest node lets the player heal a fixed percentage of max health.	3	38	
10	Rest node offers the option to gain a fixed amount of XP instead of healing.	2	39	
11	Elite nodes can be assigned on the map as an additional node type (optional).	3	56	
12	Elite nodes spawn an enemy with 1.5x the health and damage of a standard enemy (optional).	3	57	
13	Boss fight at the final node with unique attack patterns and phases.	8	43	
60	Straight-line map placeholder for Sprint 1	3	4.5	1
	Game Logic			
14	Turn-based combat loop with attack and damage resolution and win/loss conditions.	13	7	1
15	Enemies are defined as data resources (health, damage, speed, actions) so new enemies can be added without code changes.	5	9	1
16	Swordsman has a health value that is displayed in the HUD.	3	11	1
17	Turn order is determined by each combatant's speed stat.	3	12	
18	Energy system gives each skill an energy cost, lets the player use multiple skills per turn until energy runs out, and refills energy at turn start.	3	13	
19	Cooldown system locks a used skill for a set number of rounds (1 round for basic attack and block, more for stronger skills).	3	14	
20	Relic system tracks owned relics and applies their bonuses for the rest of the run.	5	34	
21	Consumable inventory stores healing items and buffs that can be used in combat.	5	40	
22	Status effect system applies, ticks, and expires effects on the player and enemies.	5	28	
23	Poison status effect deals damage at the start of each turn.	2	29	
24	Bleed status effect deals damage over time.	2	30	
25	Sleep status effect causes the afflicted combatant to skip turns until damaged.	2	48	
26	Killing enemies awards XP and gold	2	19	
27	Winning a fight has a chance to reward a healing item or buff.	2	41	
28	Reaching an XP threshold levels up the player and grants a skill point.	3	20	
	Skills			
29	Skill tree system supports unlocking nodes with prerequisites and applying their effects.	8	21	
30	Skill tree screen displays all three branches, node states (locked, available, owned), and spendable points.	5	22	

Req ID	Description	Story Pts	Priority	Sprint No.
31	Defense branch: Block skill grants a temporary shield that absorbs damage for low energy and a 1-round cooldown.	2	24	
32	Defense branch: Parry skill negates an enemy attack and counterattacks if timed to its intent.	2	51	
33	AOE branch: Rupturing Slash applies bleed to all enemies, and the bleed spreads on enemy death.	3	33	
34	AOE branch: Sweeping Slash skill damages all enemies in a fight for moderate energy and a multi-round cooldown.	2	25	
35	Big Hit branch: Heavy Strike skill deals increased damage for high energy and a multi-round cooldown.	2	23	
36	Big Hit branch: Charged Attack skill delays damage one turn for a much larger hit.	3	31	
	Enemy Characteristics			
37	Enemy intent (next action) is shown above the enemy before the player's turn.	2	26	
38	Grunt enemy with a balanced attack pattern.	2	10	
39	Brute enemy with high health that attacks once every turn.	2	27	
40	Knight enemy with high defense and a mix of attack and block actions.	2	46	
41	Archer enemy that attacks from range with weighted attack choices.	3	47	
42	Defeating an elite enemy drops a relic (optional).	3	58	
43	Elite fights award more gold than standard fights (optional).	2	59	
	Collectibles			
44	Relic: +1 to all damage dealt.	1	35	
45	Relic: +1 to Speed stat	1	44	
46	Relic: +1 to all block gained.	1	36	
	HUD			
47	Combat UI shows health bars, skill buttons with energy cost and cooldown, consumable inventory, and a turn indicator.	3	8	1
48	Healing item count is displayed in the HUD.	1	42	
49	Current floor level is displayed in the HUD.	1	49	
50	Game maintains a constant 60 FPS during map, combat, and menu screens.	3	54	
51	Current gold total is displayed in the HUD.	1	18	
52	Owned relics are displayed as icons in the HUD with a hover description.	2	50	
	UI and Menus			
53	Main menu with Start Run and Quit buttons.	2	15	1
54	Tutorial popup shows the controls when the game starts and explains map navigation and the skill tree.	3	53	
55	Death screen shows a run summary (floors cleared, gold earned, relics collected) with a Restart Run option.	3	16	1
56	Victory screen appears after the boss is defeated, with a Restart Run option.	2	17	1
57	Game is exported to HTML5 and playable in a web browser from a public link.	5	45	1
	User Experience			
58	Sound effects play for attacks, blocks, button clicks, and gold gain.	2	52	
59	Browser window resizeable to be full screen or windowed	2	32	1

Req ID	Description	Story Pts	Priority	Sprint No.
	# of Requirements	**59**		
	Total # of Story Points	**197**		

Rogue Swordsman is a turn-based fantasy rogue-like where the player defeats enemies to advance farther into the dungeon and eventually defeat a boss to win the game.

The player plays as a melee swordsman that goes through a randomly seeded, procedurally generated dungeon, encounters enemies, and uses different attacks/skills to best his foes. Defeated enemies can drop various loot like gold, XP, and even special relics that can assist the player through his journey. The player has the chance to meet shopkeepers, rest in a rest area, or if he's unlucky, encounter an elite enemy that is more challenging than the rest.

At the end of the dungeon, the swordsman will face a final boss that once defeated, will mean victory for our brave warrior.